KING SAUD UNIVERSITY COLLEGE OF COMPUTER AND INFORMATION SCIENCES Computer Science Department	
CSC 111: Computer Programming - I	**Project:** Phase#2

Parking Slot Reservation System

Description

In this phase, you will redesign your Phase 1 Parking Slot Reservation System using object-oriented programming. The system should preserve the same menus, rules, predefined data, and functionality from Phase 1, while organizing the parking slot data using a class named ParkingSlot and well-structured methods in the main class.

The system must still use five predefined parking slots and five predefined parking durations: 1 Hour, 2 Hours, 3 Hours, 4 Hours, and Day Pass. The hourly rate remains 5 SAR, and the Day Pass price remains 30 SAR.

1. ParkingSlot Class

Define a class named ParkingSlot according to the following UML:

ParkingSlot
- **slotId**: int - **available**: boolean
+ ParkingSlot() + ParkingSlot(int slotId, boolean available) + reserve(): void + isAvailable(): boolean + display(): void + setters and getters as needed

Attributes

- **slotId:** the ID of the parking slot.
- **available:** indicates whether the slot is available or occupied.

Methods

- **ParkingSlot()**
  A default constructor that initializes the slot ID to 0 and sets the parking slot as available.
- **ParkingSlot(int slotId, boolean available)**
  A parameterized constructor that initializes the parking slot using the given values.
- **reserve(): void**
  Marks the parking slot as occupied.
- **isAvailable():** boolean
  Returns true if the parking slot is available; otherwise, it returns false.

- **display(): void**
  Displays the parking slot information in a formatted row.
- **Setters and getters as needed.**

2. Main Class Methods

To further organize your program, add and use the following methods in the main class along with the ParkingSlot class.

- **displayMenu(): void**
  Displays the main menu and allows the user to access the Customer Menu, the Parking Administrator Menu, or exit the program.
- **customerMenu():void**
  Displays the customer menu and allows the customer to view available parking slots, select a parking slot and duration, proceed to checkout and generate an invoice, or cancel and return to the main menu.
- **generateInvoice(ParkingSlot slot, int duration): void**
  Generates and displays a formatted invoice for the selected parking slot and duration, including the slot ID, selected duration, and total cost. The customer must then be able to confirm or cancel the reservation.
- **administratorMenu(): void**
  Displays the administrator menu and allows the administrator to view total profit, view the most popular parking duration, or return to the main menu.
- **displayPopularDuration(): void**
  Displays the most popular parking duration based on the number of confirmed reservations, or an appropriate message if no reservations have been made.
- **allSlotsOccupied(): boolean**
  Checks whether all parking slots are occupied and returns true if no slots are available; otherwise, it returns false.

Additional Requirements and Guidelines

1. Follow the UML notations precisely.
2. All user input must be validated.
3. The program should have at least one call for each method listed in the UML and main class methods.
4. When the user needs to select a parking slot or parking duration, the system must display a menu showing all available options.
5. The program must terminate automatically when all parking slots are occupied.
6. Code must be clearly organized, properly indented, and commented.
7. At the top of the code, include a comment with the group's students' names, IDs, section number, and Lab Instructor's name.

8. Document all issues encountered during the project along with their solutions. You will be asked to discuss these challenges and explain your design decisions.
9. Only course-covered concepts may be used. Arrays are not allowed. Only String.format() may be used for formatting, and any use of external material will be graded zero.
10. Plagiarism and the use of AI tools (e.g., ChatGPT, Copilot, Gemini, Claude) are strictly prohibited and will incur severe penalties.

Submission Requirements and Rules

1. **Deadline:** April 26, 2026 at 5:00 PM
2. **Submission** (through LMS):
    - Submit the code as a **.java** file.
    - Submit a copy of the same code as a **.txt** file.